

Subject: Adv. Java

• Sec-A (5*2=10)

1. What is socket programming in java with example?

Ans Socket programming in java allows communication between two machine over a network (like the internet) by using sockets. Sockets provide the endpoint for sending and receiving data between the client and server.

2. What is Port number?

Ans A Port number is a numerical identifier in networking that allows different applications or services on a computer to be distinguished from one another. It helps direct data to the correct application or process when communicating over a network.

• Common ports:

• HTTP uses Port 80

• HTTPS uses Port 443

• FTP uses port 21

3. Write name of classes used in connection oriented and connection less communication.

Ans • Connection - Oriented (TCP):

• ServerSocket

• Socket

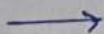
• Connection less (UDP)

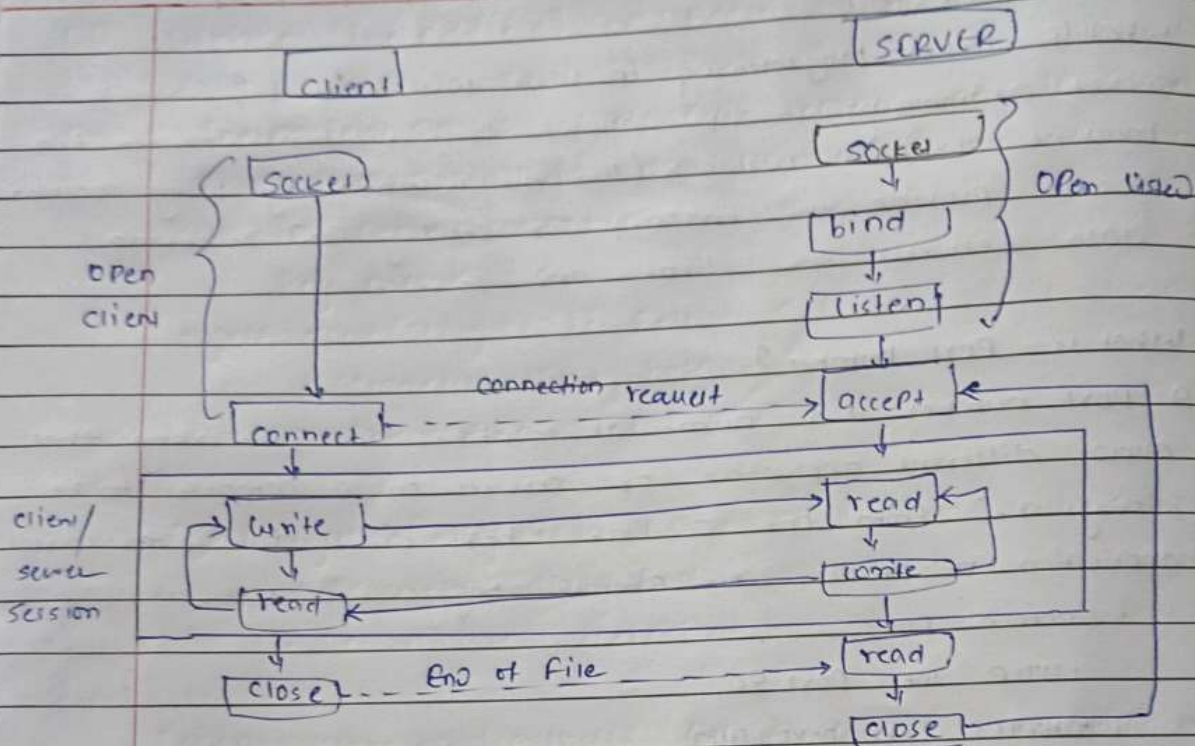
• DatagramSocket

• DatagramPacket

4. Draw the neat and clean diagram for client-server java socket programming?

Ans Diagram is in next page





SOCKET API

Q5. What is the use of openConnection method in URL class?

Ans. The openConnection() method in the URL class is used to create a connection to the specified URL. It returns a URLConnection object that can be used to read from or write to the resource identified by the URL.

• Section-B (3x7=21)

1. WAP to display the HTTP header key fields and value.

ans

```

import java.net.*;
import java.io.*;
import java.util.*;
  
```

```

public class HTTPHeaderDisp {
  
```

```

public static void main (String [] args) {
    try {

        URL url = new URL ("http://example.com");

        HttpURLConnection connection = (HttpURLConnection) url.openConnection();
        connection.setRequestMethod ("GET");
        connection.connect();

        Map<String, List<String>> headers = connection.getHeaderFields();
        for (Map.Entry<String, List<String>> entry : header.entrySet()) {
            String key = entry.getKey();
            List<String> values = entry.getValue();

            System.out.println (key + ": " + String.join ("", values));
        }
        connection.disconnect();
    } catch (Exception e) {
        s.o.p (e);
    }
}

```

Output: null: HTTP/1.1 200 OK
X-cache: HIT
Server: EC2/Ubuntu
Content-Encoding: gzip
Accept-Ranges: bytes
Age: 539407
Cache-Control: max-age=604800
last-modified: Thu, 17 Oct 2022 9:18:26 GMT
Content-Length: 648

Subject: _____

Subject: _____

Q2. What is InetAddress, Inet4Address, Inet6Address class?

Explain methods with parameter and return type.

Ans (a) InetAddress class :- InetAddress represents an IP address (both IPv4 and IPv6) and provides methods for host-to-IP address resolution.

Key methods:

• getByName(String host): Resolves a hostname to an InetAddress.

• Parameter: String host

• Return type: InetAddress

• getAllByName(String host): Returns all IP addresses for a hostname.

• Parameter: String host

• Return Type: InetAddress

• ~~getByName~~ getLocalHost(): Gets the local host's InetAddress.

• Return type: InetAddress

• getHostAddress(): Return the IP address as a string.

• Return Type - String

• isReachable(int timeout): Checks if the address is reachable within the specified timeout.

• Parameter: int timeout

• Return Type: boolean

(b) Inet4Address class :- Inet4Address is a subclass of InetAddress that specifically represents IPv4 addresses (32-bit).

• No additional methods beyond those inherited from InetAddress.

• IPv4 addresses are represented in the format like

192.168.1.1

Subject: _____

(C) Inet6Address class:- Inet6Address is a subclass of InetAddress that specifically represent IPv6 addresses (128-bit).

Specific Method:-

- isIPv4CompatibleAddress(): checks if the IPv6 address is IPv4-compatible.
- Return type: boolean

Q3. WAP of displaying details of an URL like protocol, hostname, port number, file name etc.

Ans

```
import java.net.*;

public class URLDemo {
    public static void main (String [] args) {
        try {
            URL url = new URL ("http://www.javapoint.com/Java-tutorial");
            System.out.println ("Protocol: " + url.getProtocol());
            System.out.println ("Host Name: " + url.getHost());
            System.out.println ("Port Number: " + url.getPort());
            System.out.println ("File Name: " + url.getFile());
        }
        catch (Exception e) {
            System.out.println (e);
        }
    }
}
```

Output:

Protocol : http

Host Name : www.javapoint.com

Port Number : -1

File Name : /Java-tutorial/

Subject: _____

Subject: _____

Section - C (3X10 = 30)

Q1

Ans

Write a client-server application of chatting using TCP/IP.

Server Code :-

The server accepts multiple clients and broadcasts messages to all connected clients.

import java.io.*;

import java.net.*;

import java.util.*;

public class ChatServer {

private static final int PORT = 12345;

private static Set<PrintWriter> clientWriters = new HashSet<>();

public static void main (String[] args) {

S.o.p (" Chat Server is running - ");

try (ServerSocket serverSocket = new ServerSocket (PORT)) {

while (true) {

new ClientHandler (serverSocket.accept(), start ());

}

} catch (IOException e) {

System.out.println (e);

}

}

private static class ClientHandler extends Thread {

private Socket socket;

private PrintWriter out;

private BufferedReader in;

private String clientName;

public ClientHandler (Socket socket) {

Subject: _____

```

        this.socket = socket;
    }

    public void run() {
        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);
            synchronized (clientWriters) {
                clientWriters.add(out);
            }

            out.println("Enter your name:");
            clientName = in.readLine();
            broadcast(clientName + " has joined the chat!");

            Starting message;
            while ((message = in.readLine()) != null) {
                s.o.p("Received: " + message);
                broadcast(clientName + ": " + message);
            }
        } catch (IOException e) {
            s.o.p(e);
        }
        finally {
            try {
                socket.close();
            } catch (IOException e) {
                s.o.p(e);
            }
        }

        synchronized (clientWriters) {
            clientWriters.remove(out);
            broadcast(clientName + " has left the chat!");
        }
    }

```

Subject: _____

Subject: _____

```

private void broadcast(String message) {
    synchronized (clientWriters) {
        for (PrintWriter writer : clientWriters) {
            writer.println(message);
        }
    }
}
}
}
}

```

- Client code:- The client connects to the server and allows the user to send and receive messages.

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

```

```

public class ChatClient {
    private static final String SERVER_ADDRESS = "localhost";
    private static final int SERVER_PORT = 12345;
    private PrintWriter out;

```

```

    public static void main(String[] args) {
        new ChatClient().start();
    }

    private void start() {
        try (Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT)) {
            out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(
                socket.getInputStream()));
            Scanner sc = new Scanner(System.in);

```



```

new Thread () -> {
    String message;
    try {
        while ((message = in.readLine()) != null) {
            System.out.println(message);
        }
    } catch (IOException e) {
        System.out.println(e);
    }
}

).start();

Starting newInput()
while (true) {
    newInput = sc.nextLine();
    if (newInput.equalsIgnoreCase("exit")) {
        break;
    }
    out.println(newInput);
}
} catch (IOException e) {
    System.out.println(e);
}
}
}

```

O/P:

Server O/P:	Client 1	Client 2	Broadcast
Chat server is running	Ank Your name:	Ankur Your name	Ankur has joined the chat
Received: Ankit	Ankit	Sumit	Received: Sumit
Ankit has joined	Hello everyone	Hi, Ankit	Sumit has joined the chat!
Received: Hello everyone			
Received: Hi Ankit			Ankit: Hello everyone!
			Sumit: Hi Ankit

Subject: _____

Q2 = WAP to read the content of URL. Using URLConnection
 class

Ans

```

import java.io.*;
import java.net.*;

public class URLConnection {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://www.javapoint.com/java-tutorial");
            URLConnection urlcon = url.openConnection();
            InputStream stream = urlcon.getInputStream();
            int i;
            while ((i = stream.read()) != -1) {
                System.out.print((char)i);
            }
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

O/P: <html>
 <head><title>301 Moved Permanently</title></head>
 <body>
 <center><h1>301 Moved Permanently</h1></center>
 <hr><center>Now Here</center>
 </body>
 </html>

Q3 = Write a Java program to implement Client-server application using DatagramSocket and DatagramPacket.

Subject: _____

Date: ___/___/___

Ans Sender : side (client)

```

import java.net.*;

public class Dsender {
    public static void main (String [] args) throws Exception {

        DatagramSocket ds = new DatagramSocket();
        String str = "Welcome Java";

        InetAddress ip = InetAddress.getByName ("123.0.0.1");
        DatagramPacket dp = new DatagramPacket (str.getBytes(),
            str.length(), ip, 3000);
        ds.send (dp);
        ds.close ();
    }
}

```

• Receiver (server) :-

```

import java.net.*;

public class DReceiver {
    public static void main (String [] args) throws Exception {

        DatagramSocket ds = new DatagramSocket (3000);

        byte[] buf = new byte [1024];

        DatagramPacket dp = new DatagramPacket (buf, 1024);

        ds.receive (dp);
    }
}

```


Subject: _____

```
String str = new String(dp.getData(), 0, dp.getLength());  
System.out.println(str);
```

```
ds.close();
```

```
}  
}
```

O/P: Welcome Java